MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

Final Report

Michaľ Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
office@pads.rwth-aachen.de

Abstract. Predictive process monitoring uses historical event log data to predict outcomes and remaining times of running process instances, with standard approaches deriving features from the event log alone while ignoring temporal resource-load variation. This project tests whether aligned time-series features (active-case counts with rolling-window statistics) improve prediction quality. We built a modular four-stage pipeline for the BPI Challenge 2017 loan application log, comparing log-only Random Forest models against log+time-series models for binary outcome classification (approved vs. not-approved) and remaining-time regression. The hourly-resampled time-series features with 8-hour rolling windows added no meaningful improvement for classification (F1-weighted +0.1%) and degraded regression (RMSE +1.45). We analyse why this extraction strategy fails to improve either task and discuss implications for feature engineering in predictive process monitoring using our open-source and fully reproducible pipeline.

Keywords: Predictive Process Monitoring · Process Mining · Time-Series Features · Random Forest · Model Evaluation · Explainability

1 Introduction

Predictive process monitoring (PPM) predicts future states of running process instances from event log data, with common tasks including binary outcome prediction (will this case end normally or deviate?) and remaining-time regression (how many hours until completion?). Most PPM approaches derive features from the event log only, such as activity counts, elapsed time, and case attributes, capturing what happened in a case but not the concurrent workload on the process. The BPI Challenge 2017 dataset [**?**] records a loan application process at a Dutch financial institution with validation, offer, and approval stages, where resource contention during peak periods can affect both outcomes and processing times. Log-based features alone do not capture this workload variation over time.

This project answers: *do aligned time-series workload features improve predictive performance for outcome and remaining-time tasks compared to event-log features alone?* We built a modular pipeline that extracts log-based fea-

tures, computes time-series features, trains multiple different models, and evaluates them against baselines. The contributions are: an open-source, reproducible PPM pipeline with time-series support, an empirical comparison of log-only vs. log+time-series models on both tasks, and an analysis of when time-series features help and when they do not.

The paper is organised as follows. section 2 describes the dataset and preprocessing. section 3 covers feature engineering. section 4 specifies the models and evaluation protocol. section 5 presents results. section 6 describes testing evidence. section 7 discusses findings and limitations. section 8 concludes.

2 Data and Preprocessing

2.1 Event Log: BPI Challenge 2017

The BPI Challenge 2017 event log [**?**] records a loan application process at a Dutch financial institution. Cases describe the lifecycle of credit applications, including application processing, offer creation, document validation, and customer interactions. Events belong to three categories: application state changes, offer state changes, and workflow activities performed by employees [**?**].

The dataset is widely used in predictive process monitoring as it contains rich control-flow, temporal, and financial information along with substantial variation in case duration and outcome. Table 1 summarises the dataset.

Table 1: BPI Challenge 2017 dataset statistics.

Statistic	Value
Cases	31,509
Events	1,202,267
Activities	26
Start date	2016-01-01
End date	2017-02-01
Mean case length (events)	38.16
Median case length (events)	35

2.2 Prediction Tasks

This study evaluates model performance on two predictive process monitoring tasks: loan outcome classification and remaining-time regression. Cases terminate as approved, declined, or cancelled, corresponding to the occurrence of `A_Pending`, `A_Denied`, or `A_Cancelled` respectively [**?**].

For outcome prediction, we formulate a binary classification task with approved cases as the positive class while declined and cancelled cases form a single negative class. For remaining-time prediction, the target variable is the time in hours between the last event of a prefix and case completion.

2.3 Preprocessing Pipeline

Raw XES event logs are first ingested, cleaned, and sorted in ascending order by timestamp before a sliding-window approach generates training instances. For each case, a window of length k at time step t consists of the most recent k events ending at position t, with k constrained by a minimum and maximum window length. Each resulting window is associated with the corresponding case label. Table 2 summarises the final dataset sizes for each prediction task along with the applied filtering steps and sliding-window configuration.

The preprocessing pipeline differs between the regression and classification tasks. For regression, no additional filtering is applied as sliding windows are generated with a minimum length of 3 and a maximum length of 100 events per case, producing truncated event histories that capture recent temporal context. For classification, additional constraints ensure label consistency and prevent information leakage: cases without any defined outcome events are removed (98 out of 31,509 cases), and windows containing an outcome event are excluded since their inclusion would leak information about the final case outcome into the input representation. The same minimum and maximum window lengths (3 to 100 events) are used as in the regression setup.

Table 2: Preprocessing summary per task.

Component	Value
Initial dataset	31,509 cases / 1,202,267 events
Window type	Sliding history window
Window length	min = 3, max = 100
Regression filtering	None (all cases retained)
Classification filtering	No-outcome cases removed (98 excluded), outcome prefixes removed
Regression final size	31,509 cases
Classification final size	31,411 cases

2.4 Temporal Train-Test Split

We use a temporal split by case start time to prevent leakage, ensuring all prefixes from the same case remain in the same split. Cases starting before the 0.8 quantile go to training while later cases go to testing, simulating an operational setting where the model predicts on cases that start after the training period. Table 3 shows the resulting split statistics.

Table 3: Temporal split statistics per prediction task.

	Train	Test
Classification task		
Cases	22,859	6,283
Prefixes	763,417	210,096
Split timestamp	2016-10-18 11:39:41	
Regression task		
Cases	22,895	6,302
Prefixes	827,614	228,455
Split timestamp	2016-10-18 22:39:59	

3 Feature Engineering

3.1 Log-Based Features (Baseline)

Log-based features are derived purely from the event log, with Table 4 listing the feature groups. Activity counts are encoded per event type while temporal features use the timestamp of the last event in the prefix, with all features computed in vectorised form for efficiency.

Table 4: Log-based feature groups.

Group	Features	Count
Activity counts	Count per event type per prefix	26
Temporal	Elapsed time, time since last event	2
Financial	Monthly cost, number of terms	2
Waiting states	Time in waiting activities	4
Total		34

3.2 Time-Series Features (Enhanced)

Time-series features capture concurrent process workload at hourly resolution, where for every hourly base signal we compute three features: the raw value, an 8-hour trailing mean (`rolling_mean_8h`), and a trend (`trend_8h` = current value minus value 8 hours earlier). Table 5 lists the time-series feature classes.

Table 5: Time-series feature classes.

Class	Base signal	Base cols	Total
ActiveCaseCount	Cases active at prefix timestamp	1	3
FinancialVolume	Hourly sum/mean of withdrawal, offer, cost	6	18
DecisionRate	Ratio of accepted decisions per hour	1	3
Total		8	24

3.3 Feature Extraction Pipeline

Features are extracted via a modular pipeline where each feature set implements the `PrefixFeature` protocol (`name()`, `fit()`, `__call__()`, `feature_names`), and the feature extractor generates a single feature matrix by concatenating all enabled feature sets. Time-series features are precomputed once during `fit()` and aligned to each prefix at extraction time via vectorised timestamp lookup.

4 Model Specification and Evaluation Protocol

4.1 Prediction Targets

Table 6 shows the class distributions for the train and test splits at both trace and prefix level, with the distributions highly consistent between sets and indicating a stable split without noticeable sampling bias. At trace level, a moderate class imbalance towards approved cases can be observed, becoming more pronounced at prefix level where approved cases form a larger proportion of the data.

Table 6: Class distribution for traces and prefixes (train/test split).

Level	Split		Count	Percentage (%)
Traces	Train	approved	12,694	55.5%
		not approved	10,165	44.5%
	Test	approved	3,449	54.9%
		not approved	2,834	45.1%
Prefixes	Train	approved	516,099	67.6%
		not approved	247,318	32.4%
	Test	approved	141,171	67.2%
		not approved	68,925	32.8%

Table 7 shows remaining-time statistics at prefix level, with the target distribution highly right-skewed as indicated by the substantial difference between mean and median values in both train and test sets. The mean remaining time

is approximately 300 hours while the median is considerably lower (around 205 hours), indicating that many prefixes occur relatively late in the process where little time remains until case completion, whereas a smaller number of early prefixes contribute disproportionately large remaining-time values. This leads to a long upper tail in the distribution with maximum values exceeding 3000 hours in the training set, while the interquartile range further reflects this variability with 50% of prefix instances lying approximately between 40 and 480 hours remaining.

The train and test distributions are highly consistent across all statistics, suggesting that the temporal structure of prefixes is preserved across the split with no significant distribution shift.

Table 7: Distribution of remaining time (in hours) for train and test prefix sets.

Statistic	Train	Test
Prefixes	827,614	228,455
Mean	302.8	300.0
Median	205.1	208.8
Std	315.5	298.1
Min	0	0
Max	3269.0	2348.1
Q1	42.3	45.3
Q3	479.4	488.8

4.2 Models and Hyperparameter Optimisation

We evaluate four model families for both prediction tasks (Table 8), covering linear baselines and non-linear ensemble methods. The selected models are widely used in predictive process monitoring due to their ability to handle high-dimensional, sparse, and heterogeneous event-based feature representations.

Table 8: Models and hyperparameter search spaces.

Model	Task	Key hyperparameters
Random Forest	Both	`n_estimators`: 100–300 `max_depth`: 5–20 `min_samples_split`: 2–20
Logistic Regression	Classification	`C`: 0.01–100
Ridge Regression	Regression	`alpha`: 0.01–100
HistGradientBoosting	Both	`learning_rate`: 0.01–0.3 `max_iter`: 100–300

Hyperparameter optimisation is performed using `RandomizedSearchCV` with 3-fold cross-validation on the training set. All models are trained exclusively on training prefixes, and evaluation is performed on a held-out test set.

Table 9 lists the search configuration.

Table 9: Hyperparameter optimisation configuration.

Parameter	Value
Search method	RandomizedSearchCV
CV folds	3
Search iterations	10
Search sample size	All training prefixes

4.3 Evaluation Protocol and Comparison Design

For outcome classification, we report three complementary evaluation metrics. **F1-weighted** serves as the primary ranking metric, as it balances precision and recall while accounting for class imbalance. **ROC AUC** provides a threshold-independent measure of class discrimination and is widely used in predictive process monitoring literature. **Matthews Correlation Coefficient (MCC)** is additionally reported as a balanced metric that incorporates all entries of the confusion matrix and remains informative under class imbalance.

For remaining-time prediction, we use **RMSE** (in hours) as the primary ranking metric, since it directly reflects prediction error in the target unit and penalises large deviations more strongly. In addition, R^2 is reported to quantify the proportion of variance explained relative to a mean-value baseline.

To identify the most suitable predictive model for each task, all four candidate models are trained and evaluated using the full feature set consisting of log-based and time-series features. The best-performing model is then selected according to the primary ranking metric (F1-weighted for classification and RMSE for regression). To quantify the contribution of time-series information, the selected model is additionally trained and evaluated using only log-based features. Both configurations use the same train/test split, hyperparameter optimisation procedure, and evaluation pipeline. The resulting full-feature and log-only variants are then compared to assess the predictive value of the time-series features.

5 Results

The best model differs by task: HistGradientBoosting (HGB) extended with time-series features achieves the highest F1-weighted score for classification (0.6601), while Random Forest (RF) with just log-based features achieves the lowest RMSE for regression (233.14).

5.1 Outcome Prediction

HGB outperforms all alternatives for outcome classification, achieving the highest scores across all three primary metrics (Table 10). HGB leads Random Forest by 2.1% in F1-weighted (0.6601 vs. 0.6465), by 0.006 in ROC AUC, and by 0.019 in MCC.

Table 10: Classification model comparison (log+TS, 58 features, full data).

Model	F1-weighted	ROC AUC	MCC
HistGradientBoosting	**0.6601**	**0.6580**	**0.2316**
Random Forest	0.6465	0.6522	0.2128
Logistic Regression	0.6297	0.6270	0.1827
Dummy (most frequent)	0.5401	0.5000	0.0000

Figure 1 shows F1-weighted across prefix lengths, with performance improving consistently as more events become available: HGB rises from 0.5051 at prefix 3 to 0.7731 at prefix 34. The improvement rate diminishes after the first 10–15 events, but longer prefixes continue to add signal.

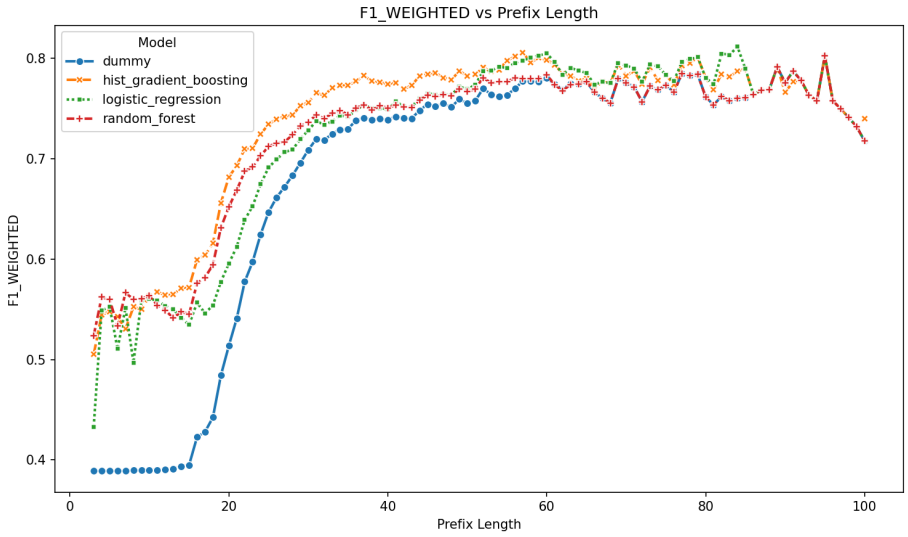

Fig. 1: F1-weighted by prefix length (classification).

Time-Series Impact Adding hourly-resampled TS features to HGB shows no significant changes. With 34 log-only features, HGB scores 0.6584 F1-weighted

and 0.6597 ROC AUC. With 58 log+TS features, it scores 0.6594 and 0.6557 (Table 11). The differences (+0.001 in F1-weighted and −0.004 in ROC AUC) are negligible.

Table 11: Classification TS impact (HGB, full data).

Configuration	Features	F1-weighted	ROC AUC
HGB log-only (34)	34	0.6584	0.6597
HGB log+TS (58)	58	0.6594	0.6557

Feature Importance To interpret model predictions, feature importance was estimated using permutation importance. For each feature, its values are randomly shuffled while all other features remain unchanged. The resulting decrease in model performance is recorded as the feature's importance score. Features that cause a larger performance degradation when permuted are considered more influential for the model's predictions. [**?**]

Table 12 presents the ten most important features for the best-performing classification model (HGB, log-only features). The feature `count_W_Validate application` is by far the most influential predictor, with an importance score of 0.0636. Its contribution is approximately three times larger than that of the second-ranked feature, `count_W_Handle leads`, indicating that the occurrence of validation activities contains a substantial portion of the predictive signal. Several additional workflow and financial features also contribute to model performance, although with considerably smaller importance scores.

Table 12: Top 10 features by permutation importance (HGB log-only).

Feature	Importance
count_W_Validate application	0.0636
count_W_Handle leads	0.0209
monthly_cost	0.0123
num_terms	0.0097
elapsed_time_hours	0.0074
count_A_Validating	0.0061
count_O_Returned	0.0060
count_W_Call after offers	0.0057
count_W_Call incomplete files	0.0043
time_since_last_event_hours	0.0023

Figure 2 visualises the importance distribution, with activity-count features occupying 6 of the top 10 positions while financial and temporal features appear

at middle ranks with substantially lower importance and waiting-state features appearing nowhere in the top 10.

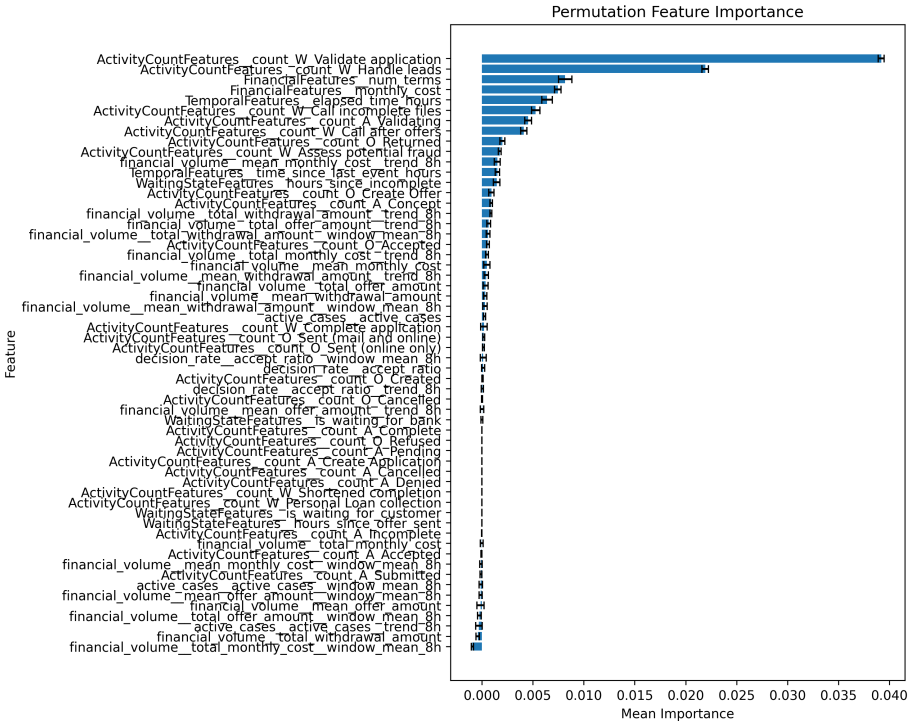

Fig. 2: Permutation feature importance (HGB log-only).

Figure 3 shows the SHAP summary dot plot for the same model, where high occurrences of `count_W_Validate application` push predictions toward approved while higher `elapsed_time_hours` and `time_since_last_event_hours` push toward not-approved, meaning long-running cases tend to end negatively.

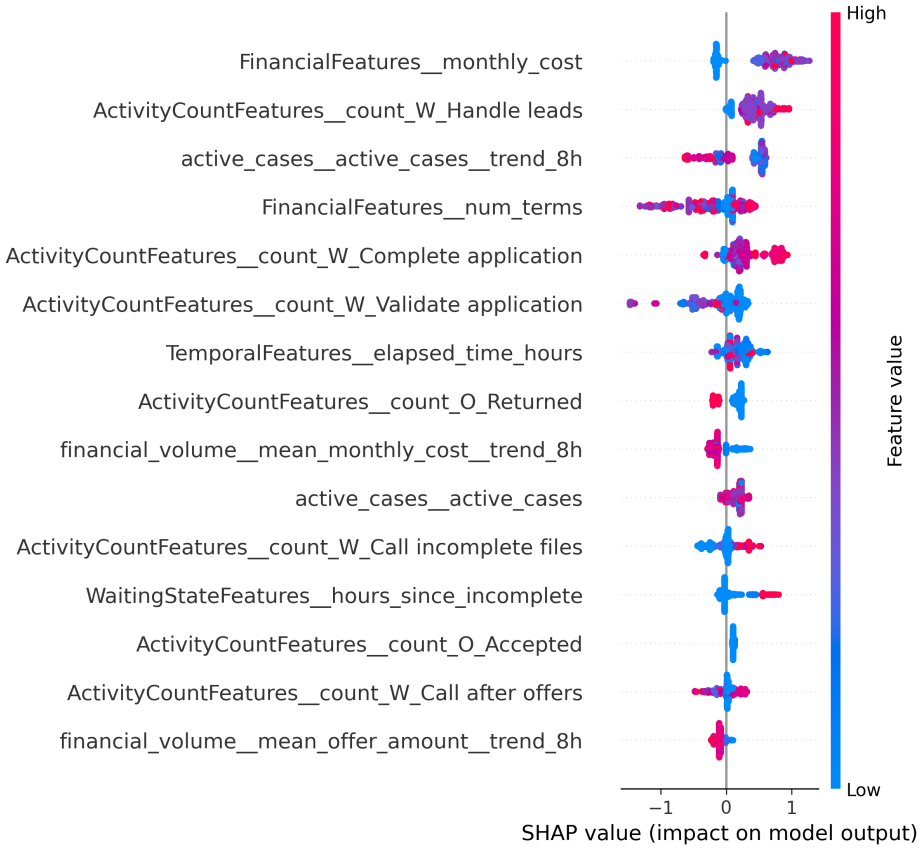

Fig. 3: SHAP summary dot plot (HGB log-only classification). Each dot represents a single prediction. Colour indicates feature value (red = high, blue = low). Features are ordered by global importance.

5.2 Remaining-Time Prediction

Using the full feature set (log-only + time-series), Random Forest (RF) achieves the lowest RMSE for remaining-time regression, marginally ahead of HGB and well below the dummy baseline (Table 13). The R^2 of 0.382 indicates that RF explains 38% of the variance in remaining time.

Table 13: Regression model comparison (log+TS, max_length=100, full data).

Model	RMSE (hours)	R^2
Random Forest	**234.43**	0.382
HistGradientBoosting	234.88	0.379
Ridge	240.87	0.347
Dummy (mean)	298.13	0.000

Figure 4 shows RMSE across prefix lengths as it improves from 284.83 at prefix 3 to 233.14 at prefix 100, with most of the gain achieved by prefix 20. Beyond that, returns diminish but do not fully plateau.

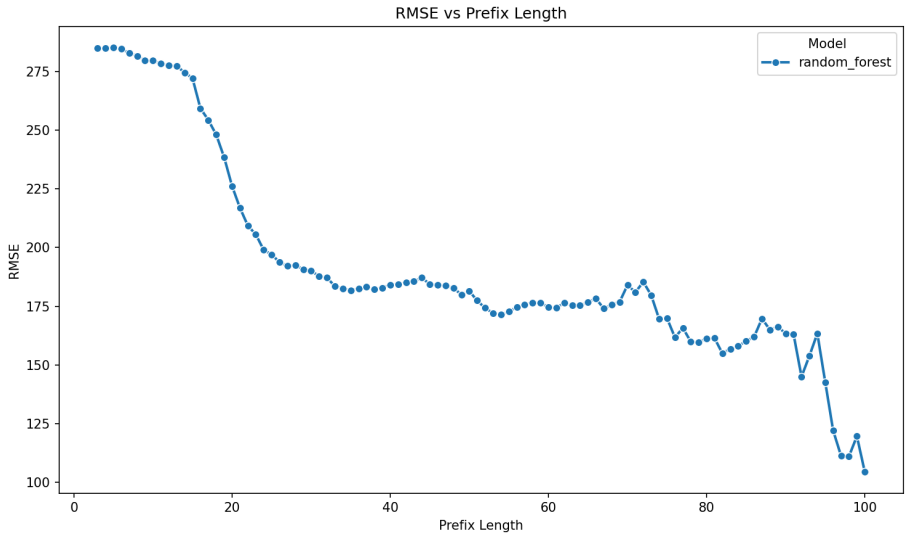

Fig. 4: RMSE by prefix length (RF log-only regression).

Time-Series Impact Time-series features degrade regression performance slightly. The log-only RF (233.14 RMSE) beats the log+TS variant by 1.42 RMSE (Table 14), a 0.6% penalty. The degradation is consistent across all prefix lengths.

Table 14: Regression TS impact (RF, max_length=100, full data).

Configuration	Features	RMSE (hours)
RF log-only	34	**233.14**
RF log+TS	58	234.56

Full-Length vs. Maximum-Length Prefixes Capped (`max_length=100`) and unbounded (full trace) prefix configurations produce nearly identical results at 234.43 vs. 234.37 RMSE, with a difference of 0.04% indicating that longer histories do not help.

Feature Importance Feature Importance is computed the same way as for classification (see subsection 5.1). Table 15 lists the top 10 features. Top is `count_W_Validate application` (0.0618), then `count_W_Call after offers` (0.0323) and `count_A_Cancelled` (0.0299). Activity counts fill 7 of the top 10. Temporal features are less important.

Table 15: Top 10 features by permutation importance (RF log-only).

Feature	Importance
count_W_Validate application	0.0618
count_W_Call after offers	0.0323
count_A_Cancelled	0.0299
count_A_Validating	0.0132
elapsed_time_hours	0.0131
count_O_Returned	0.0127
count_A_Complete	0.0111
time_since_last_event_hours	0.0096
count_O_Accepted	0.0068
num_terms	0.0048

5.3 Overfitting Analysis

Table 16 reports train-test gaps for the best models while Figure 5 visualises the comparison.

Table 16: Train-test performance gap.

Task	Model	Train	Test	Gap
Classification	HGB (f1-weighted)	0.6971	0.6609	−0.0362
Regression	RF log-only (RMSE)	248.08	233.14	−14.94

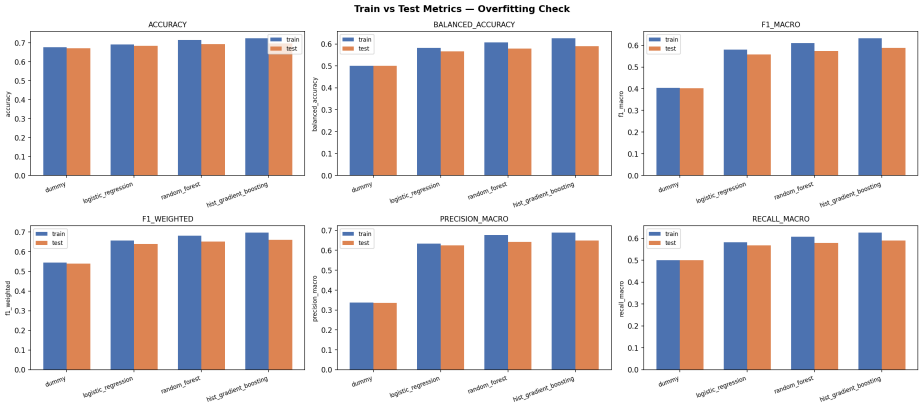

Fig. 5: Train vs. Test performance (all models log-based + time-series) for classification.

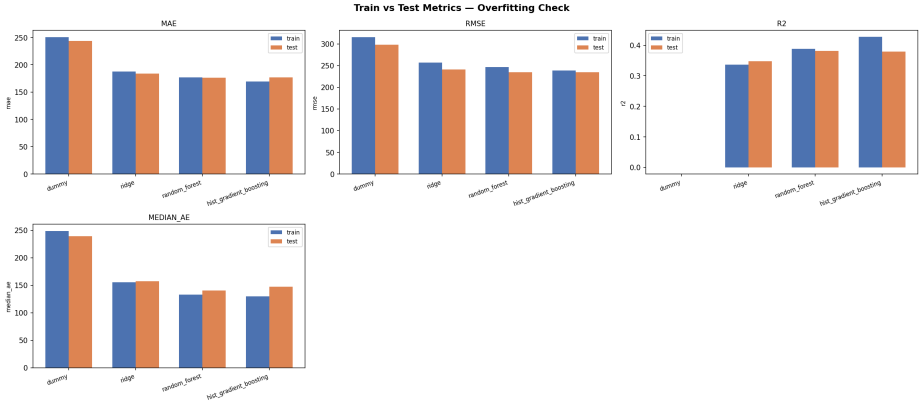

Fig. 6: Train vs. Test performance (all models log-based + time-series) for regression.

Overfitting is moderate with the train-test gap for HGB at −0.036 F1-weighted and for RF regression at −14.94 RMSE (6% of the test RMSE), both within expected bounds for tree-based ensemble models without explicit regularisation.

5.4 Summary of Findings

– **HGB is best for classification** (0.6601 F1-weighted, 0.6580 ROC AUC, 0.2316 MCC), outperforming RF by 2.1%.

- **RF is best for regression** (233.14 RMSE log-only, R^2=0.382), marginally ahead of HGB.
- **Time-series features add no significant value** for classification (+0.1% F1-weighted) and **harm** regression (+1.45 RMSE).
- **Log-only RF (233.14 RMSE)** beats all log+TS variants
- **count_W_Validate** is the single most predictive feature across both tasks (importance 0.0636 classification, 0.0618 regression). The most important features for both tasks are ActivityCount features.
- Performance improves with longer prefixes, but plateaus after 25 events.
- Overfitting gaps are moderate (HGB −0.036, RF −14.94 RMSE).

6 Testing Evidence

The project includes 133 unit and integration tests across 11 test modules. Tests run on small synthetic event logs whose expected outputs can be verified by inspection, favouring meaningful behavioural checks over file-existence checks. The breakdown by module is:

- **Targets (10)**: Remaining-time and outcome-label construction on synthetic logs, including the full-prefix remaining-time-zero case and outcome priority / first-match rules.
- **Preprocessing (10)**: Sliding-window and outcome-window generation, and temporal-split integrity.
- **Log-based features (14)**: Activity counts, temporal, financial, and waiting-state features with shape, naming, and edge cases.
- **Time-series features (2)**: Derived-column construction (raw, 8-hour mean, 8-hour trend) and backward as-of alignment to prefixes.
- **Dataset (7)**: Download, extraction, caching, and HTTP error handling (1 marked `integration`).
- **Metrics (35)**: MAE, RMSE, R^2, F1, ROC AUC, balanced accuracy, and `compare_models` weighted-averaging correctness.
- **Plots (11)**: Task detection, primary-metric selection, and figure generation (metric-vs-prefix, error distribution, predicted-vs-actual, and ROC/PR curves).
- **SHAP (5)**: Classification and regression explanations, with a graceful skip for non-tree models.
- **Trainer (19)**: Model fitting, artifact structure, pipeline steps, prediction, the optional PCA step, and hyperparameter-search smoke tests.
- **Config (20)**: Schema validation, estimator construction, YAML round-trip, and error handling (loader and schema modules).

All tests pass with a single command:

```
poetry run pytest
```

The one network-dependent dataset test can be excluded for a fully offline run with `poetry run pytest -m "not integration"`.

7 Critical Evaluation

7.1 Do Time-Series Features Improve Predictive Performance?

For classification, the hourly-resampled time-series features with 8-hour rolling windows added no meaningful improvement (F1-weighted +0.1%). For regression, they consistently degraded performance (RMSE +1.45), with the log-only model outperforming all log+TS variants. No single cause is decisive; the most likely explanation is a combination of the factors below, which concern the nature of the signals, how they were extracted and aligned, how they interact with the chronological split, and how they affect the models.

Concept drift / covariate shift (the strongest candidate). The log spans roughly a year, and the chronological split places earlier cases in train and later cases in test. The time-series features are global process-state signals that are inherently non-stationary in calendar time, so *their train and test distributions differ substantially* - far more than the case-relative log features, whose values (activity counts within a prefix, elapsed time since case start) do not depend on when in the year the case ran. The pipeline's two-sample Kolmogorov–Smirnov feature-drift evaluation (`feature_drift_stats.csv`) flags exactly this shift for the workload columns. A feature whose distribution moves between train and test cannot generalise: a tree ensemble that fits the training regime of `active_cases` and its rolling statistics is then evaluated against a test regime that no longer matches, which is the most plausible mechanism behind the active-case-count block degrading weighted R^2 on the majority of prefix lengths and behind the time-series block failing to help either task. The effect is sharper for regression, where the target is itself time-related, than for the more stable classification task.

The signals were never validated as informative. We compute `active_cases`, hourly financial volumes, and an hourly acceptance ratio, but never independently verify that any of them tracks known busy periods or correlates with the target. The null result may therefore reflect a poor operationalisation of "workload" rather than a property of time-series information itself.

Global signals, local contention. `ActiveCaseCountFeature` is a single global count of cases in progress, and `FinancialVolumeFeature` and `DecisionRateFeature` are likewise global hourly aggregates. The contention that plausibly affects a given case is the load on the specific resource or queue handling *that* case; a global, log-wide count averages this away, so even a genuine workload effect would be largely invisible at the resolution we measure it.

Proxy effect: the log features already encode load. Log-based features (activity counts, elapsed time, case-level financial attributes) may already carry indirect workload information. The dominance of `count_W_Validate application`, a gate activity, across both tasks (Table 12, Table 15) is consistent with this: case maturity and queue position are partly observable from the control flow alone, leaving little marginal signal for the time-series block to add.

Coarse hourly resolution and an 8-hour window. The mean case lasts ∼302 hours, so the 8-hour window covers only ∼2.6% of a typical case. `window_mean_8h` and `trend_8h` cannot see the daily (24h) and weekly (168h) cycles that dominate office-hours loan processing. Worse, `trend_8h` is a first-order difference (current value minus the value 8 hours earlier), which amplifies stochastic noise rather than measuring a stable directional change.

A narrow feature template. Every base signal receives the same raw / 8-hour-mean / 8-hour-trend triplet. Standard recent lags (t-1h, t-2h), seasonal lags (t-24h, t-168h), and explicit calendar encodings (hour-of-day, day-of-week) were all omitted, despite strong priors for daily and weekly seasonality. The time-series block is thus a single narrow operationalisation rather than a representative sample of time-series features.

Hourly binning and zero-filling inject artefacts. Aggregating to a complete hourly grid and filling empty hours with 0 conflates "no events this hour" with "a genuinely low value", and discards any within-hour structure. For `FinancialVolumeFeature` in particular, quiet hours and zero-activity hours become indistinguishable.

Backward-looking alignment answers the wrong question. Each prefix is matched to the most recent *past* hourly bucket via a backward as-of lookup (`searchsorted` / `merge_asof`, `direction="backward"`). The features therefore describe the load the case has *already* experienced, whereas remaining time and final outcome depend on the load it will encounter *next*. The signal is leakage-safe but points the wrong way in time relative to the prediction target.

Right-censoring contamination. End-of-log right-censoring distorts both the active-case time series (cases still open near the end of the log are mis-counted) and the regression target in the test period. This corrupts the signal precisely in the late-calendar, long-prefix regime where any genuine time-series contribution would be most visible.

Added dimensionality, mostly redundant. The enhanced set adds 24 columns to the 34 log-based features. The raw / mean / trend triplet is mechanically collinear, and several derived columns are near-constant. Irrelevant and collinear features are known to dilute the splits available to tree ensembles and slightly raise variance, which is consistent with the small but consistent regression RMSE penalty rather than an improvement.

Synthesis. Taken together, these factors make the answer on this log, with this operationalisation, a clear no. The result is specific to the chosen signals, their hourly resolution and short window, the backward alignment, and the non-stationarity introduced by the chronological split; it is not evidence that process workload is irrelevant to outcome or remaining time in principle. Stationary or detrended workload features, finer resolution, seasonal lags, and per-resource load remain plausible routes that this study did not test.

7.2 Limitations

Possible reasons the time-series features did not help are discussed in subsection 7.1. This section collects the broader methodological limitations that bound how far any of our conclusions generalise.

Single dataset. All results come from one event log (BPIC 2017). The findings - the null effect of the time-series block and the dominance of the `count_W_Validate application` gate activity - may not transfer to processes with different control-flow structure, load patterns, or target distributions. We make no claim of external validity beyond this log.

Model scope. We evaluate linear baselines and two tree ensembles (Random Forest, HistGradientBoosting). Axis-aligned tree splits consume each time-series column independently and cannot model temporal dependence across lags; sequence models (LSTMs, temporal CNNs, transformers) integrate continuous time-series information differently and might extract signal the trees cannot. The negative time-series result is therefore specific to this model class.

Limited hyperparameter search. Hyperparameter optimisation used `RandomizedSearchCV` with a small budget (10 iterations, 3-fold CV), and the families were not tuned to equal depth. Reported differences between model families may partly reflect search budget rather than intrinsic capability; on a low-R^2, elapsed-time-dominated target the families largely converge, so feature signal, not model choice, is the binding constraint.

Evaluation protocol. A single deterministic temporal split was used. A rolling-origin / time-series cross-validation would give a more robust estimate and reveal variance across calendar periods, which one split cannot. Hyperparameters and feature sets were fixed rather than selected on a dedicated validation split; this avoids tuning toward the test set, but means feature selection was never exercised.

Metrics. The classifiers reach MCC < 0.25 throughout, so the task is hard and margins between configurations are small. F1-weighted, the primary ranking metric, is sensitive to the 67:33 prefix-level prevalence and rewards the majority class; ROC AUC and MCC are reported alongside it to compensate.

Feature attribution. Permutation importance is biased when features are correlated, and the time-series triplet (raw, 8-hour mean, 8-hour trend) is mechanically correlated within itself and with the log-based temporal features, so the importance rankings are indicative only. Group-level ablation and TreeSHAP are the more defensible tools for comparing the log-based and time-series feature blocks.

8 Conclusion

8.1 Summary

This project investigated whether aligned time-series workload features improve predictive process monitoring. We built a modular pipeline for the BPI Challenge 2017 loan application log, extracting 34 log-based features and 24 hourly-resampled time-series features (raw value, 8-hour rolling mean, and 8-hour trend per signal), and compared four model families per task. HistGradientBoosting achieved the almost identical classification performance with and without time-series features (0.6601 F1-weighted, 0.6580 ROC AUC, 0.2316 MCC with time-series), while Random Forest achieved the lowest regression error (233.14 RMSE, log-only).

The time-series features added no meaningful value for classification (+0.1% F1-weighted) and slightly degraded regression (+1.45 RMSE): the log-only models matched or beat every log+TS variant on both tasks. The gate activity `count_W_Validate application` dominated feature importance across both tasks.

We attribute the null result primarily to concept drift: the time-series features are global, non-stationary process-state signals whose distributions shift across the chronological train/test split, so they fail to generalise and even penalise the drift-sensitive regression target. A proxy effect - log-based activity counts already encoding case maturity and queue position - and the backward-looking alignment of the workload signals are potentially contributing factors. The practical implication is that, on this log and with this operationalisation, practitioners can prioritise log-based features, particularly counts of gate activities, before investing in time-series feature engineering; this is not evidence that workload is irrelevant in principle, only that raw, non-stationary workload signals did not help here. The pipeline and all experiments are open-source and reproducible from a single configuration.

8.2 Project Retrospective

This section reflects on the work as a software project rather than on its empirical findings, which are covered in section 7.

What worked. The modular pipeline architecture met its design goal. Separating `config`, `data`, `preprocessing`, `features`, `models`, and `evaluation` made it cheap to add the HistGradientBoosting family late in the project and to toggle feature blocks through configuration alone, without touching pipeline code. Stage-level checkpointing kept the iteration loop fast once preprocessing was stable. The single-command test suite and YAML-driven experiments meant that every reported number is tied to a committed configuration, making the log-only vs. log+TS comparison reproducible by construction.

What did not work. The central hypothesis was not confirmed: the time-series features added no value for classification and slightly harmed regression. As an engineering outcome this is a clean negative result, but it reflects a misallocation of effort: the time-series block was built and integrated before anyone verified that its signals tracked real load variation or shared a distribution across the train/test split. The drift that ultimately explained the null result (subsection 7.1) was diagnosed late, using a feature-drift check that would have been more valuable as an upfront gate than as a post-hoc explanation.

Requirement evolution. The high-level requirements remained stable, but the emphasis of the deliverable shifted. The project was framed as demonstrating a benefit from time-series features; as evidence accumulated, it became an analysis of *why* that benefit failed to materialise. Detailed sub-requirements were managed as GitHub issues linked to their parent requirement rather than by editing the requirements document, which kept the specification stable while still recording scope changes such as the addition of the HistGradientBoosting family and the decision to make PCA an optional, off-by-default pipeline step.

Technical learnings. Three lessons recurred. First, concept drift and covariate shift on a chronological split are first-order concerns: non-stationary signals that look predictive in training can degrade test performance, which is exactly what the active-case-count block did. Second, end-of-log right-censoring contaminates both the time-series feature and the regression target in the test period, and must be reasoned about explicitly rather than discovered in the metrics. Third, feature signal, not model choice, was the binding constraint: RF and HGB converged on a low-R^2, elapsed-time-dominated target, so further model tuning would not have moved the result.

Organisational learnings. The agile sprint structure with GitHub issues worked well for a three-person team and gave a usable audit trail from requirement to code to report. The main friction was integration timing: feature, modelling, and evaluation work proceeded somewhat in parallel, so some inconsistencies surfaced only when the full comparison was assembled.

The planned role redistribution between sprints was largely ignored by the team. With only three members and a small, manageable project, modular specialisation proved to be an unnecessary overhead. Our workflow required cross-module knowledge to apply even small changes or integrate new functionality. Furthermore, given our university schedules, we often worked asymmetrically, meaning everyone needed to be able to continue and review work done by others.

What we would do differently. We would gate every candidate time-series signal on an upfront validity and drift check before modelling; add recent and seasonal lags (t-1h, t-24h, t-168h) and detrended workload signals given the strong daily/weekly priors and the observed non-stationarity; perform feature selection on a dedicated validation split rather than the final test set; profile runtime from

the start rather than at the end; and add a smoke check that every configured feature actually fires on the real log, which would have caught the activity-matching bug immediately.

A Environment Setup and Reproduction

Source and data. Source code: https://github.com/durki007/spi-time-series. The BPI Challenge 2017 log [?] is downloaded automatically on first run.

Prerequisites. Python 3.12 and Poetry. Install dependencies with:

```
poetry install
poetry install --with test
```

Running the experiments.

```
poetry run python -m spi_time_series.main \
    experiments/classification/classification_compare.yaml

poetry run python -m spi_time_series.main \
     experiments/regression/regression_compare.yaml

poetry run python -m spi_time_series.main \
    experiments/regression/regression_rf_log.yaml

poetry run python -m spi_time_series.main \
    experiments/classification/classification_hgb_log.yaml
```

Each run writes metrics and figures (including `roc_pr_curves.png`) and saves the trained pipeline state to `checkpoint.joblib` in the output directory.

Tests.

```
poetry run pytest
```

B Feature Descriptions

This appendix specifies every feature, its computation, unit, and edge-case behaviour. All features are extracted per prefix and emitted as `float32`. A prefix is the ordered array of events up to a cutoff position; the "last event" below always refers to the final event of the prefix.

B.1 Log-Based Features (34 features)

ActivityCountFeatures (26). A bag-of-activities encoding: one integer count per activity type in the fixed vocabulary `EVENT_NAMES` (26 activities), computed as the number of occurrences of that activity within the prefix (e.g. `count_W_Validate application`, `count_A_Accepted`). Activities absent from the prefix receive a count of 0, so the block is fixed-width and order-independent across prefixes. Counts are unbounded above and grow with prefix length, which is why they dominate the importance rankings for both tasks.

TemporalFeatures (2). Two elapsed-time measures derived from event timestamps, both in hours:

- `elapsed_time_hours`: time between the first and last event of the prefix, i.e. how long the case has been running at the cutoff.
- `time_since_last_event_hours`: time between the last two events of the prefix, a measure of recent inter-event idle time.

Both are set to 0 when the prefix contains a single event (no interval is defined). The hour conversion divides the raw timestamp difference by one hour.

FinancialFeatures (2). Two case-level offer attributes carried on the events:

- `monthly_cost`: the `MonthlyCost` of the offer.
- `num_terms`: the `NumberOfTerms` of the offer.

Each is taken as the *last valid* value when scanning the prefix backwards from the most recent event, skipping empty, null, or non-numeric entries. If no valid value exists in the prefix (e.g. no offer has been created yet), the feature defaults to 0. `monthly_cost` is a currency amount and `num_terms` a count.

WaitingStateFeatures (4). Features capturing whether and how long the case has been waiting at the cutoff:

- `hours_since_offer_sent`: hours between the last occurrence of an offer-sent event and the prefix cutoff; 0 if none has occurred. The matcher keys on the literal activity label `O_Sent`.
- `hours_since_incomplete`: hours since the last `A_Incomplete` event; 0 if none has occurred.
- `is_waiting_for_customer`: binary flag, 1 if the last activity is one of `O_Sent`, `A_Complete`, or `A_Incomplete` (states where the bank awaits the customer), else 0.
- `is_waiting_for_bank`: binary flag, 1 if the last activity is one of `A_Submitted`, `A_Concept`, `A_Accepted`, `A_Validating`, or `O_Returned` (states where the customer awaits the bank), else 0.

The two duration features and the customer-waiting flag match the exact label `O_Sent`; in the BPIC 2017 log the offer-sent activity is recorded as `O_Sent (mail and online)` / `O_Sent (online only)`, so these components do not fire on the real data (cf. section 7), leaving the waiting-state block largely inert and consistent with its absence from the importance rankings.

B.2 Time-Series Features (24 features)

Time-series features describe the concurrent process workload around the prefix rather than the case itself. They are precomputed once over the full cleaned log during `fit()` and aligned to each prefix at extraction time. Every base signal expands into three columns, giving 24 features from 8 base signals.

Extraction Mechanics (shared by all classes).

- **Hourly resampling.** Each base signal is aggregated onto a complete hourly grid spanning the log (`pd.date_range(..., freq="1h")`). Hours with no underlying events are filled with 0 so the grid has no gaps.
- **Derived columns.** For each base column `<c>`, two derived columns are added: `<c>__window_mean_8h`, the trailing 8-hour rolling mean (`rolling("8h").mean()`), and `<c>__trend_8h`, a first-order difference equal to the current value minus the value 8 hours earlier (`<c> - <c>.shift(8)`). The raw, mean, and trend columns form the triplet per signal.
- **Missing-value handling.** Undefined leading values of the rolling mean and trend (the first 8 hours) are back-filled and then zero-filled, so the feature matrix contains no missing values.
- **Alignment.** At extraction each prefix is matched to the most recent hourly bucket at or before its last-event timestamp via a backward as-of lookup (`searchsorted`/`merge_asof`, `direction="backward"`); prefixes earlier than the first bucket take the first bucket. Alignment is therefore strictly backward-looking, which is leakage-safe but assigns each prefix the *past* workload value rather than the load the case will subsequently experience.

Feature Classes.

- **ActiveCaseCountFeature (1 base, 3 total).** Base signal `active_cases`: the number of cases in progress at each hour, computed from per-case start/end times as a running balance ($+1$ at each case start, -1 at each case end, cumulative sum) sampled onto the hourly grid. Expanded to `active_cases`, `active_cases__window_mean_8h`, `active_cases__trend_8h`. This block was found to degrade weighted R^2 on most prefix lengths under the chronological split (section 7).
- **FinancialVolumeFeature (6 base, 18 total).** Hourly financial throughput, grouping events by floored hour and aggregating six base columns: the sum and mean of `FirstWithdrawalAmount`, `OfferedAmount`, and `MonthlyCost` (`total_withdrawal_amount`, `mean_withdrawal_amount`, `total_offer_amount`, `mean_offer_amount`, `total_monthly_cost`, `mean_monthly_cost`). Each base column expands to its triplet, giving 18 features. Amounts are in currency units; hours with no financial events contribute 0.
- **DecisionRateFeature (1 base, 3 total).** Base signal `accept_ratio`: per hour, the number of accepted decisions (`A_Accepted`) divided by the total number of terminal decision events that hour (accepted, denied, cancelled

(`A_Cancelled`/`O_Cancelled`), and pending (`A_Pending`)). Hours with no decisions yield a ratio of 0. The ratio lies in $[0, 1]$ and expands to `accept_ratio`, `accept_ratio__window_mean_8h`, `accept_ratio__trend_8h`.